

Project Title: Claims Processing & Policy Management System

Business Domain: Insurance

1. Overview

The Claims Processing & Policy Management System manages insurance operations end to end, including policy issuance, premium collection, claim intake, adjudication, settlement, and fraud detection. Built using an MVC architecture, it supports both Java (Spring MVC) and .NET (ASP.NET Core MVC) frameworks.

The system consists of five key modules:

1. Policy Issuance & Underwriting
2. Premium Collection & Renewal Management
3. Claim Intake & Document Verification
4. Claim Adjudication & Settlement
5. Fraud Detection & Investigation Tracking

2. Assumptions

- **Database:** MySQL or SQL Server.
- **Roles:** Underwriter, Policy Administrator, Claims Adjuster, Fraud Investigator, Admin.
- **ORM:** Hibernate (Java), Entity Framework Core (.NET).
- Payment gateway integration is simulated.
- Fraud scoring uses mock rule-based engines.

3. Module-Level Design

3.1 Policy Issuance & Underwriting Module

Purpose: Captures new policy applications and underwriting decisions.

Controller:

- PolicyController
 - createPolicy()
 - performUnderwriting()

- getPolicyDetails()
- issuePolicy()

Service:

- PolicyService
 - Applies underwriting rules, risk classification, and premium calculation.

Model:

- Policy Entity
 - Attributes:
 - policyId (PK)
 - policyHolderName
 - policyType
 - sumInsured
 - premiumAmount
 - policyStatus (PROPOSED, ISSUED, LAPSED, CANCELLED)

3.2 Premium Collection & Renewal Management Module

Purpose: Tracks premium payments and renewal cycles.

Controller:

- PremiumController
 - recordPremiumPayment()
 - scheduleRenewal()
 - getPremiumHistory()

Service:

- PremiumService
 - Manages payment schedules, grace periods, and renewal notifications.

Model:

- PremiumPayment Entity

- Attributes:
 - paymentId (PK)
 - policyId (FK)
 - paymentDate
 - amountPaid
 - paymentMode
 - paymentStatus (RECEIVED, PENDING, FAILED)

3.3 Claim Intake & Document Verification Module

Purpose: Captures claim notifications and supporting documents.

Controller:

- ClaimIntakeController
 - registerClaim()
 - uploadClaimDocument()
 - getClaimStatus()

Service:

- ClaimIntakeService
 - Validates claim eligibility and document completeness.

Model:

- Claim Entity
 - Attributes:
 - claimId (PK)
 - policyId (FK)
 - incidentDate
 - claimAmount
 - claimType
 - claimStatus (REGISTERED, UNDER_REVIEW, APPROVED, REJECTED, SETTLED)

3.4 Claim Adjudication & Settlement Module

Purpose: Evaluates claims and processes settlement payouts.

Controller:

- AdjudicationController
 - reviewClaim()
 - approveClaim()
 - processSettlement()

Service:

- AdjudicationService
 - Applies policy terms, exclusions, and computes payable amounts.

Model:

- Settlement Entity
 - Attributes:
 - settlementId (PK)
 - claimId (FK)
 - approvedAmount
 - settlementDate
 - payoutMode
 - settlementStatus (PENDING, PAID, ON_HOLD)

3.5 Fraud Detection & Investigation Tracking Module

Purpose: Flags suspicious claims and manages investigation lifecycle.

Controller:

- FraudController
 - flagSuspiciousClaim()
 - assignInvestigator()
 - logInvestigationFinding()

Service:

- FraudService
 - Runs rule-based fraud scoring and tracks investigation outcomes.

Model:

- FraudCase Entity
 - Attributes:
 - caseId (PK)
 - claimId (FK)
 - fraudScore
 - investigatorId
 - findingSummary
 - caseStatus (OPEN, INVESTIGATING, CLEARED, CONFIRMED_FRAUD)

4. Database Schema

Policy Table

```
CREATE TABLE Policy (  
  policyId INT AUTO_INCREMENT PRIMARY KEY,  
  policyHolderName VARCHAR(100),  
  policyType VARCHAR(50),  
  sumInsured DECIMAL(12,2),  
  premiumAmount DECIMAL(10,2),  
  policyStatus ENUM('PROPOSED', 'ISSUED', 'LAPSED', 'CANCELLED')  
);
```

PremiumPayment Table

```
CREATE TABLE PremiumPayment (  
  paymentId INT AUTO_INCREMENT PRIMARY KEY,  
  policyId INT,  
  paymentDate DATE,  
  amountPaid DECIMAL(10,2),  
  paymentMode VARCHAR(50),  
  paymentStatus ENUM('RECEIVED', 'PENDING', 'FAILED'),  
  FOREIGN KEY (policyId) REFERENCES Policy(policyId)  
);
```

Claim Table

```
CREATE TABLE Claim (  
  claimId INT AUTO_INCREMENT PRIMARY KEY,  
  policyId INT,  
  claimAmount DECIMAL(10,2),  
  claimStatus ENUM('OPEN', 'CLOSED', 'PENDING'),  
  FOREIGN KEY (policyId) REFERENCES Policy(policyId)  
);
```

```
claimId INT AUTO_INCREMENT PRIMARY KEY,  
policyId INT,  
incidentDate DATE,  
claimAmount DECIMAL(12,2),  
claimType VARCHAR(50),  
claimStatus ENUM('REGISTERED','UNDER_REVIEW','APPROVED','REJECTED','SETTLED'),  
FOREIGN KEY (policyId) REFERENCES Policy(policyId)  
);
```

Settlement Table

```
CREATE TABLE Settlement (  
settlementId INT AUTO_INCREMENT PRIMARY KEY,  
claimId INT,  
approvedAmount DECIMAL(12,2),  
settlementDate DATE,  
payoutMode VARCHAR(50),  
settlementStatus ENUM('PENDING','PAID','ON_HOLD'),  
FOREIGN KEY (claimId) REFERENCES Claim(claimId)  
);
```

FraudCase Table

```
CREATE TABLE FraudCase (  
caseId INT AUTO_INCREMENT PRIMARY KEY,  
claimId INT,  
fraudScore INT,  
investigatorId INT,  
findingSummary VARCHAR(255),  
caseStatus ENUM('OPEN','INVESTIGATING','CLEARED','CONFIRMED_FRAUD'),  
FOREIGN KEY (claimId) REFERENCES Claim(claimId)  
);
```

5. Local Deployment Details

- Install DB + JDK 17/21/24 or .NET SDK 8.0
- Deploy using Tomcat or Kestrel

Steps:

1. Clone repository.
2. Run SQL schema.
3. Configure app settings files.
4. Build & run the project.

6. Conclusion

The Claims Processing & Policy Management System provides a robust MVC-based structure for insurance lifecycle operations, claims adjudication, and fraud control. It is ideal for training scenarios and local deployment.